Sanskruti Sarees — Full API Contract

Base URL: `/api/v1`
Auth: Customer endpoints use Bearer JWT token. Admin endpoints use a separate Admin Bearer JWT token. Public endpoints require no token.
Format: All request and response bodies use `application/json` unless marked `multipart/form-data`.

Global response shape

Success

```
{
  "success": true,
  "message": "Human-readable success message",
  "data": {},
  "meta": {}
}
```

Error

```
{
  "success": false,
  "message": "Human-readable error message",
  "errors": {
    "field_name": ["Validation message"]
  }
}
```

HTTP status codes used

Code	Meaning
200	OK
201	Created
400	Bad request or business rule violation
401	Not authenticated
403	Authenticated but not authorized
404	Resource not found
409	Conflict (e.g. duplicate review)
422	Validation failed
500	Internal server error

Global validation rules

- All UUID fields must be valid UUID v4.
- All price fields must be non-negative decimals up to 10 digits with 2 decimal places.
- All weight fields must be positive integers.
- All timestamp fields must be ISO 8601 format.
- All slug fields must be lowercase, hyphenated, and alphanumeric only.

- All string fields are trimmed; empty strings treated as null unless required.

Section 1 — Public: Homepage

`GET /api/v1/home`

Auth: Public
Purpose: Returns all homepage sections in one call for the homepage render.

Query params: None

Response `data`**:**


```
{
  "announcement_strip": {
    "text": "Free shipping in Gujarat!",
    "is_active": true
  },
  "hero_slides": [
    {
      "hero_slide_id": "uuid",
      "image_url": "cdn-url",
      "heading": "New Collection",
      "subtext": "Explore premium sarees",
      "button_label": "Shop Now",
      "target_url": "/sarees",
      "display_order": 1
    }
  ],
  "main_banners": {
    "saree": {
      "image_url": "cdn-url",
      "heading": "Sarees",
      "button_label": "Shop Sarees",
      "target_url": "/sarees"
    },
    "chaniya_choli": {
      "image_url": "cdn-url",
      "heading": "Chaniya Cholis",
      "button_label": "Shop Chaniya Cholis",
      "target_url": "/chaniya-cholis"
    }
  },
  "new_arrivals": [
    {
      "product_id": "uuid",
      "product_name": "Banarasi Silk Saree",
      "product_slug": "banarasi-silk-saree",
      "cover_image_url": "cdn-url",
      "selling_price_inr": 4500.00,
      "original_price_inr": 5500.00
    }
  ],
  "marketing_banners": [
    {
      "marketing_banner_id": "uuid",
      "image_url": "cdn-url",
      "title": "Festive Offers",
      "description": "Up to 20% off on wedding sarees",
      "button_label": "Shop Offer",
      "target_url": "/collections/festive",
      "display_order": 1
    }
  ],
  "shop_by_fabrics": [
    {
      "fabric_id": "uuid",
```

```
      "fabric_name": "Silk",
      "fabric_slug": "silk",
      "cover_image_url": "cdn-url"
    }
  ],
  "premium_products": [
    {
      "product_id": "uuid",
      "product_name": "Kanjivaram Silk Saree",
      "product_slug": "kanjivaram-silk-saree",
      "cover_image_url": "cdn-url",
      "selling_price_inr": 12000.00
    }
  ],
  "shop_by_collections": [
    {
      "collection_id": "uuid",
      "collection_name": "Wedding",
      "collection_slug": "wedding",
      "cover_image_url": "cdn-url"
    }
  ],
  "whatsapp_info": {
    "whatsapp_number": "+919723421411",
    "message": "Need help? Chat with us on WhatsApp for product questions, order support, and cancellat
  }
}
```


Errors:

- 500 — server error fetching homepage data.

Section 2 — Public: Product Listings

`GET /api/v1/products`

Auth: Public
Purpose: Paginated product list with filters and sorting. Works for both Sarees and Chaniya Cholis.

Query params:

Param	Type	Required	Notes
`category_slug`	string	No	`sarees` or `chaniya-cholis`
`search`	string	No	Full-text search on product name
`page`	integer	No	Default 1
`limit`	integer	No	Default 20, max 100
`sort`	string	No	`newest`, `price_low_to_high`, `price_high_to_low`. Default `newest`
`min_price`	decimal	No	Min selling price filter
`max_price`	decimal	No	Max selling price filter
`fabric_ids`	CSV string	No	e.g. `uuid1,uuid2`
`collection_ids`	CSV string	No	e.g. `uuid1,uuid2`
`color_ids`	CSV string	No	e.g. `uuid1,uuid2`
`availability`	string	No	`in_stock` or `out_of_stock`
`tag_normal`	boolean	No	Saree filter

Param	Type	Required	Notes
`has_ready_blouse`	boolean	No	Saree filter
`is_bridal_chaniya_choli`	boolean	No	CC filter
`is_sider_chaniya_choli`	boolean	No	CC filter
`filter_{filter_type_slug}`	CSV string	No	Dynamic filter values e.g. `filter_work_type=zari,handwork`

Validation rules:

- `page` must be integer >= 1.
- `limit` must be integer between 1 and 100.
- `sort` must be one of allowed values.
- `min_price` and `max_price` must be non-negative decimals; `min_price` must not exceed `max_price`.
- `fabric_ids`, `collection_ids`, `color_ids` must be valid UUIDs if provided.
- Dynamic filter params are resolved by matching `filter_type_slug` against active filter types and then filtering by `filter_value_slug` values.

Response `data`:

```
{
  "items": [
    {
      "product_id": "uuid",
      "product_name": "Banarasi Silk Saree",
      "product_slug": "banarasi-silk-saree",
      "cover_image_url": "cdn-url",
      "selling_price_inr": 4500.00,
      "original_price_inr": 5500.00,
      "fabric_name": "Silk",
      "collection_name": "Wedding",
      "color_name": "Red",
      "color_hex_code": "#b00020",
      "is_in_stock": true,
      "tags": ["with_ready_blouse"]
    }
  ]
}
```

Response `meta`:

```
{
  "total_items": 120,
  "total_pages": 6,
  "current_page": 1,
  "limit": 20,
  "available_filters": {
    "fabrics": [],
    "collections": [],
    "colors": [],
    "dynamic_filters": []
  }
}
```

Errors:

- `422` — invalid query param values.

`GET /api/v1/products/sarees`

Auth: Public
Purpose: Saree-specific listing. Internally calls the same logic as `/api/v1/products` with `category_slug=sarees` preset.

Accepts same query params as `/api/v1/products` minus `category_slug`.

`GET /api/v1/products/chaniya-cholis`

Auth: Public
Purpose: Chaniya Choli listing with `category_slug=chaniya-cholis` preset.

Accepts same query params as `/api/v1/products` minus `category_slug`.

`GET /api/v1/products/:product_slug`

Auth: Public
Purpose: Full product detail page data.

Path params:

Param	Type	Required
`product_slug`	string	Yes

Response `data`**:**

```
{
  "product_id": "uuid",
  "product_name": "Kanjivaram Silk Saree",
  "product_slug": "kanjivaram-silk-saree",
  "product_description": "...",
  "product_care_instructions": "...",
  "selling_price_inr": 12000.00,
  "original_price_inr": 15000.00,
  "available_stock_quantity": 5,
  "is_in_stock": true,
  "weight_in_grams": 600,
  "category": { "category_id": "uuid", "category_name": "Saree" },
  "fabric": { "fabric_id": "uuid", "fabric_name": "Silk" },
  "collection": { "collection_id": "uuid", "collection_name": "Wedding" },
  "color": { "color_id": "uuid", "color_name": "Red", "color_hex_code": "#b00020" },
  "tags": {
    "is_saree_normal": true,
    "has_ready_blouse": false,
    "is_bridal_chaniya_choli": false,
    "is_sider_chaniya_choli": false
  },
  "dynamic_attributes": [
    { "filter_type_name": "Work Type", "filter_value_name": "Zari" }
  ],
  "media": [
    {
      "product_media_id": "uuid",
      "media_url": "cdn-url",
      "media_type": "image",
      "display_order": 1,
      "is_cover_media": true,
      "alt_text": "Kanjivaram Silk Saree front view"
    }
  ],
  "finishing_promise": "Before dispatch, every saree and chaniya choli is carefully finished and qualit
  "seo_meta_title": "...",
  "seo_meta_description": "...",
```

```
  "related_products": [
    {
      "product_id": "uuid",
      "product_name": "...",
      "product_slug": "...",
      "cover_image_url": "cdn-url",
      "selling_price_inr": 9800.00
    }
  ],
  "review_summary": {
    "average_rating": 4.3,
    "total_reviews": 12
  },
  "reviews": [
    {
      "review_id": "uuid",
      "rating_value": 5,
      "review_title": "Beautiful saree",
      "review_text": "...",
      "customer_name": "Priya S.",
      "created_at": "2026-05-10T10:00:00Z"
    }
  ]
}
```

Errors:

- `404` — product not found or not active.

Section 3 — Public: Search

`GET /api/v1/search`

Auth: Public
Purpose: Search products and return suggestions.

Query params:

Param	Type	Required
q	string	Yes, min 2 chars

Validation: `q` must be at least 2 characters.

Response `data`**:**

```
{
  "products": [],
  "suggestions": ["Banarasi", "Bandhani", "Bridal Saree"]
}
```

Errors:

- `422` — `q` too short.

Section 4 — Public/Customer: Auth

`POST /api/v1/auth/register`

Auth: Public
Purpose: Customer registration.

Request body:

```
{
  "full_name": "Priya Shah",
  "email_address": "priya@example.com",
  "phone_number": "9876543210",
  "password": "StrongPass@123"
}
```

Validation rules:

- `full_name`: required, string, 2–100 chars.
- `email_address`: required, valid email format.
- `phone_number`: required, valid Indian mobile number, 10 digits.
- `password`: required, min 8 chars, must include at least one uppercase, one lowercase, one digit.

Response `data`**:**

```
{
  "user_id": "uuid",
  "full_name": "Priya Shah",
  "email_address": "priya@example.com",
  "token": "jwt-token"
}
```

Errors:

- `409` — email or phone already registered.
- `422` — validation failed on any field.

`POST /api/v1/auth/login`

Auth: Public
Purpose: Customer login.

Request body:

```
{
  "email_or_phone": "priya@example.com",
  "password": "StrongPass@123"
}
```

Validation rules:

- `email_or_phone`: required, string.
- `password`: required, string.

Response `data`**:**

```
{
  "user_id": "uuid",
  "full_name": "Priya Shah",
  "token": "jwt-token"
}
```

Errors:

- `401` — invalid credentials.
- `403` — account deactivated.
- `422` — missing fields.

`POST /api/v1/auth/logout`

Auth: Customer-auth
Purpose: Invalidate session/token.

Request body: None

Response `data`**:** `{}`

`GET /api/v1/auth/me`

Auth: Customer-auth
Purpose: Get current customer profile.

Response `data`**:**

```
{
  "user_id": "uuid",
  "full_name": "Priya Shah",
  "email_address": "priya@example.com",
  "phone_number": "9876543210",
  "is_verified": true,
  "created_at": "2026-01-01T00:00:00Z"
}
```

Errors:

- `401` — not authenticated.

Section 5 — Customer-auth: Profile and Addresses

`GET /api/v1/account/profile`

Auth: Customer-auth
Purpose: Get profile details.

Response `data`**:** Same shape as `GET /api/v1/auth/me`.

`PATCH /api/v1/account/profile`

Auth: Customer-auth
Purpose: Update profile details.

Request body (all optional):

```
{
  "full_name": "Priya Shah",
  "phone_number": "9876543210"
}
```

Validation rules:

- `full_name`: optional, 2–100 chars.

- `phone_number`: optional, 10-digit Indian mobile.

Errors:

- `409` — new phone already in use by another account.
- `422` — validation failed.

`GET /api/v1/account/addresses`

Auth: Customer-auth
Purpose: List all saved addresses.

Response `data`**:**

```
{
  "addresses": [
    {
      "address_id": "uuid",
      "recipient_full_name": "Priya Shah",
      "recipient_phone_number": "9876543210",
      "address_line_1": "12, Rose Apartments",
      "address_line_2": "Near City Mall",
      "city_name": "Surat",
      "state": { "state_id": "uuid", "state_name": "Gujarat", "state_code": "GJ" },
      "postal_code": "395001",
      "country_code": "IN",
      "is_default_address": true
    }
  ]
}
```

`POST /api/v1/account/addresses`

Auth: Customer-auth
Purpose: Add new address.

Request body:

```
{
  "recipient_full_name": "Priya Shah",
  "recipient_phone_number": "9876543210",
  "address_line_1": "12, Rose Apartments",
  "address_line_2": "Near City Mall",
  "city_name": "Surat",
  "state_id": "uuid",
  "postal_code": "395001",
  "country_code": "IN",
  "is_default_address": true
}
```

Validation rules:

- `recipient_full_name`: required, 2–100 chars.
- `recipient_phone_number`: required, 10-digit number.
- `address_line_1`: required, max 200 chars.
- `address_line_2`: optional, max 200 chars.
- `city_name`: required, max 100 chars.
- `state_id`: required, valid UUID, must exist in `states` table.
- `postal_code`: required, 6-digit Indian PIN or valid international format.

- `country_code`: required, 2-letter ISO code.
- `is_default_address`: optional boolean, default false.

Business rule: If `is_default_address = true`, all other addresses for this customer are set to `false`.

Errors:

- `404` — `state_id` not found.
- `422` — validation failed.

`PATCH /api/v1/account/addresses/:address_id`

Auth: Customer-auth
Purpose: Update address. Same fields as POST, all optional.

Errors:

- `403` — address does not belong to requesting customer.
- `404` — address not found.
- `422` — validation failed.

`DELETE /api/v1/account/addresses/:address_id`

Auth: Customer-auth
Purpose: Remove saved address.

Errors:

- `403` — address does not belong to requesting customer.
- `404` — address not found.
- `400` — cannot delete address currently used in an open/processing order.

Section 6 — Customer-auth: Wishlist

`GET /api/v1/wishlist`

Auth: Customer-auth
Purpose: Get wishlist items.

Response `data`**:**

```
{
  "items": [
    {
      "wishlist_item_id": "uuid",
      "product_id": "uuid",
      "product_name": "...",
      "product_slug": "...",
      "cover_image_url": "cdn-url",
      "selling_price_inr": 4500.00,
      "is_in_stock": true
    }
  ]
}
```

`POST /api/v1/wishlist`

Auth: Customer-auth
Purpose: Add product to wishlist.

Request body:

```
{ "product_id": "uuid" }
```

Validation rules:

- `product_id`: required, valid UUID, product must be active.

Errors:

- `404` — product not found.
- `409` — product already in wishlist.

`DELETE /api/v1/wishlist/:product_id`

Auth: Customer-auth
Purpose: Remove product from wishlist.

Errors:

- `404` — product not in wishlist.

Section 7 — Customer-auth: Cart

`GET /api/v1/cart`

Auth: Customer-auth
Purpose: Get current cart with totals.

Response `data`**:**

```
{
  "items": [
    {
      "cart_item_id": "uuid",
      "product_id": "uuid",
      "product_name": "...",
      "product_slug": "...",
      "cover_image_url": "cdn-url",
      "selling_price_inr": 4500.00,
      "quantity": 2,
      "line_total_inr": 9000.00,
      "is_in_stock": true,
      "available_stock_quantity": 5
    }
  ],
  "subtotal_inr": 9000.00,
  "item_count": 2
}
```

`POST /api/v1/cart/items`

Auth: Customer-auth
Purpose: Add item to cart.

Request body:

```
{
  "product_id": "uuid",
  "quantity": 2
}
```

Validation rules:

- `product_id`: required, valid UUID, product must be active and in stock.
- `quantity`: required, integer, min 1, must not exceed available stock.

Errors:

- `400` — out of stock or insufficient stock.
- `404` — product not found.
- `409` — product already in cart (frontend should use PATCH to update quantity).
- `422` — validation failed.

`PATCH /api/v1/cart/items/:cart_item_id`

Auth: Customer-auth
Purpose: Update cart item quantity.

Request body:

```
{ "quantity": 3 }
```

Validation rules:

- `quantity`: required, integer, min 1, must not exceed available stock.

Errors:

- `400` — insufficient stock for requested quantity.
- `403` — cart item does not belong to requesting customer.
- `404` — cart item not found.

`DELETE /api/v1/cart/items/:cart_item_id`

Auth: Customer-auth
Purpose: Remove one item from cart.

Errors:

- `403` — cart item does not belong to requesting customer.
- `404` — cart item not found.

`DELETE /api/v1/cart`

Auth: Customer-auth
Purpose: Clear entire cart.

`GET /api/v1/cart/suggestions`

Auth: Customer-auth
Purpose: "You may also like" suggestions based on cart products (same fabric/collection).

Response `data`**:**

```
{
  "suggestions": []
}
```

Section 8 — Customer-auth: Checkout and Payments

`POST /api/v1/checkout/shipping-quote`

Auth: Customer-auth
Purpose: Calculate shipping charge before placing order.

Request body:

```
{ "address_id": "uuid" }
```

Validation rules:

- `address_id`: required, valid UUID, must belong to requesting customer.

Business rule:

1. Resolve `state_id` from address and look up `states.state_code`.
2. If `state_code = GJ` and `country_code = IN` → `shipping_zone = gujarat`, `shipping_charge_inr = 0`.
3. Else if `country_code = IN` → `shipping_zone = india`, compute from active India shipping rate using total weight of all cart items.
4. Else → `shipping_zone = international`, compute from active international rate.
5. Apply `minimum_shipping_charge_inr` if computed amount is lower.

Response `data`**:**

```
{
  "shipping_zone": "gujarat",
  "total_weight_in_grams": 1200,
  "shipping_charge_inr": 0.00,
  "calculation_note": "Free shipping — Gujarat delivery"
}
```

Errors:

- `400` — no active shipping rate found for the zone.
- `400` — cart is empty.
- `404` — address not found.

`GET /api/v1/checkout/summary`

Auth: Customer-auth
Purpose: Full checkout summary: cart items, totals, and computed shipping.

Query params:

Param	Type	Required
`address_id`	UUID	Yes

Response `data`**:**

```
{
  "items": [],
  "subtotal_inr": 9000.00,
  "shipping_zone": "india",
  "shipping_charge_inr": 120.00,
  "grand_total_inr": 9120.00,
  "finishing_note": "Before dispatch, every saree and chaniya choli is carefully finished and quality-c
}
```

`POST /api/v1/checkout/create-order`

Auth: Customer-auth
Purpose: Create a pending order and a Razorpay order before payment.

Request body:

```
{
  "address_id": "uuid",
  "chaniya_choli_measurements": [
    {
      "product_id": "uuid",
      "chaniya_choli_waist_size": "28 inches",
      "chaniya_choli_length": "40 inches"
    }
  ]
}
```

Validation rules:

- `address_id`: required, valid UUID, must belong to requesting customer.
- `chaniya_choli_measurements`: required only if cart contains Chaniya Choli items.
- `chaniya_choli_waist_size`: required for CC items, string max 50 chars.
- `chaniya_choli_length`: required for CC items, string max 50 chars.
- Cart must not be empty.
- All cart items must be in stock at the time of order creation.

Business rule: Measurements are saved at order item level, not product level.

Response `data`**:**

```
{
  "order_id": "uuid",
  "order_number": "SS-20260001",
  "razorpay_order_id": "order_xyz",
  "amount_in_paise": 912000,
  "currency": "INR",
  "customer_prefill": {
    "full_name": "Priya Shah",
    "email": "priya@example.com",
```

```
    "phone": "9876543210"
  },
  "shipping_charge_inr": 120.00,
  "grand_total_inr": 9120.00
}
```

Errors:

- `400` — cart is empty.
- `400` — one or more items out of stock.
- `404` — address not found.
- `422` — missing CC measurements for CC items.

`POST /api/v1/payments/razorpay/verify`

Auth: Customer-auth
Purpose: Verify Razorpay payment signature and finalize order.

Request body:

```
{
  "order_id": "uuid",
  "razorpay_order_id": "order_xyz",
  "razorpay_payment_id": "pay_abc",
  "razorpay_signature": "signature_string"
}
```

Validation rules:

- All four fields required.
- `razorpay_signature` is verified server-side using HMAC SHA256.

Business rule: On successful verification, set `order_status = placed` and `payment_status = paid`, then clear the customer's cart.

Response `data`**:**

```
{
  "order_id": "uuid",
  "order_number": "SS-20260001",
  "order_status": "placed",
  "payment_status": "paid"
}
```

Errors:

- `400` — signature verification failed.
- `404` — order not found.

Section 9 — Customer-auth: Orders

`GET /api/v1/account/orders`

Auth: Customer-auth
Purpose: Order history with filters.

Query params:

Param	Type	Required
order_status	string	No
start_date	date	No
end_date	date	No
page	integer	No
limit	integer	No

Response `data`**:**

```
{
  "orders": [
    {
      "order_id": "uuid",
      "order_number": "SS-20260001",
      "order_status": "delivered",
      "payment_status": "paid",
      "order_grand_total_inr": 9120.00,
      "item_count": 2,
      "created_at": "2026-05-01T10:00:00Z"
    }
  ]
}
```

`GET /api/v1/account/orders/:order_id`

Auth: Customer-auth
Purpose: Full order detail.

Response `data`**:**

```
{
  "order_id": "uuid",
  "order_number": "SS-20260001",
  "order_status": "delivered",
  "payment_status": "paid",
  "shipping_zone": "gujarat",
  "shipping_charge_inr": 0.00,
  "order_subtotal_inr": 9000.00,
  "order_grand_total_inr": 9000.00,
  "delivered_at": "2026-05-10T15:00:00Z",
  "shipping_address": {},
  "items": [
    {
      "order_item_id": "uuid",
      "product_id": "uuid",
      "product_name": "Kanjivaram Silk Saree",
      "cover_image_url": "cdn-url",
      "ordered_quantity": 1,
      "unit_selling_price_inr": 9000.00,
      "line_total_price_inr": 9000.00,
      "chaniya_choli_waist_size": null,
      "chaniya_choli_length": null
    }
  ],
  "cancellation_note": "For cancellation within 48 hours of delivery, contact us on WhatsApp with your
}
```

Errors:

- `403` — order does not belong to requesting customer.
- `404` — order not found.

Section 10 — Customer-auth: Reviews

`POST /api/v1/products/:product_id/reviews`

Auth: Customer-auth
Purpose: Create a new review.

Path params:

Param	Type	Required
`product_id`	UUID	Yes

Request body:

```
{
  "order_item_id": "uuid",
  "rating_value": 5,
  "review_title": "Beautiful saree",
  "review_text": "The quality is excellent and packaging was great."
}
```

Validation rules:

- `order_item_id`: required, valid UUID.
- `rating_value`: required, integer between 1 and 5.
- `review_title`: optional, max 150 chars.
- `review_text`: required, min 10 chars, max 2000 chars.

Business rule checks (all must pass):

1. `order_item_id` must exist and belong to the requesting customer.
2. The parent order must have `order_status = delivered`.
3. No existing row in `reviews` for same `(customer_user_id, product_id)`.
4. No existing row in `review_blocks` for same `(customer_user_id, product_id)`.

Errors:

- `400` — order item not delivered yet.
- `403` — order item does not belong to requesting customer.
- `403` — review blocked due to approved cancellation.
- `404` — order item or product not found.
- `409` — review already exists for this product by this customer.
- `422` — validation failed.

Section 11 — Admin-auth: Auth

`POST /api/v1/admin/auth/login`

Auth: Public
Purpose: Admin login.

Request body:

```
{
  "email_address": "admin@sanskrutisakrees.com",
```

```
  "password": "AdminPass@123"
}
```

Validation rules:

- `email_address`: required, valid email.
- `password`: required, string.

Response `data`:

```
{
  "admin_user_id": "uuid",
  "full_name": "Admin",
  "token": "admin-jwt-token"
}
```

Errors:

- `401` — invalid credentials.
- `403` — account deactivated.

`POST /api/v1/admin/auth/logout`

Auth: Admin-auth

`GET /api/v1/admin/auth/me`

Auth: Admin-auth

Section 12 — Admin-auth: Dashboard and Reports

`GET /api/v1/admin/dashboard`

Auth: Admin-auth
Purpose: KPI metrics and chart data.

Query params:

Param	Type	Required	Notes
`range`	string	No	`today`, `last_7_days`, `last_30_days`, `custom`. Default `last_7_days`
`start_date`	date	No	Required if `range=custom`
`end_date`	date	No	Required if `range=custom`

Validation rules:

- If `range = custom`, both `start_date` and `end_date` must be provided.
- `start_date` must not be after `end_date`.

Response `data`:

```
{
  "total_orders": 42,
  "total_revenue_inr": 382500.00,
  "average_order_value_inr": 9107.14,
  "orders_over_time": [
    { "date": "2026-06-07", "order_count": 6 }
  ],
  "revenue_over_time": [
```

```
    { "date": "2026-06-07", "revenue_inr": 54000.00 }
  ]
}
```

`GET /api/v1/admin/reports/products`

Auth: Admin-auth
Purpose: Top products by revenue and quantity.

Query params: `range`, `start_date`, `end_date`, `page`, `limit`

`GET /api/v1/admin/reports/fabrics`

Auth: Admin-auth
Purpose: Top fabrics by order revenue and quantity.

`GET /api/v1/admin/reports/collections`

Auth: Admin-auth
Purpose: Top collections.

`GET /api/v1/admin/reports/cancellations`

Auth: Admin-auth
Purpose: Products with highest cancellations.

Section 13 — Admin-auth: Master Data CRUD

Fabrics

`GET /api/v1/admin/fabrics`

Auth: Admin-auth

Query params: `page`, `limit`, `search`, `is_active`

Response `data`**:**

```
{
  "fabrics": [
    {
      "fabric_id": "uuid",
      "fabric_name": "Silk",
      "fabric_slug": "silk",
      "cover_image_url": "cdn-url",
      "display_order": 1,
      "show_on_homepage": true,
      "is_active": true
    }
  ]
}
```

`POST /api/v1/admin/fabrics`

Request body (multipart/form-data):

Field	Type	Required	Notes
`fabric_name`	string	Yes	2–100 chars
`fabric_slug`	string	Yes	Unique slug
`cover_image_file`	file	Yes	Image from admin device only
`display_order`	integer	No	Default 0
`show_on_homepage`	boolean	No	Default false
`is_active`	boolean	No	Default true

Errors:

- `409` — slug already in use.
- `422` — validation failed.

`PATCH /api/v1/admin/fabrics/:fabric_id`

Same fields as POST, all optional. `cover_image_file` accepted as new upload.

`DELETE /api/v1/admin/fabrics/:fabric_id`

Errors:

- `400` — fabric is used by active products; cannot delete.

Collections, Colors

Same CRUD shape as Fabrics. Collections have `collection_name`, `collection_slug`, `cover_image_file`. Colors have `color_name`, `color_hex_code`, no image.

Section 14 — Admin-auth: Dynamic Filters

`GET /api/v1/admin/filter-types`

Auth: Admin-auth

Response `data`**:**


```
{
  "filter_types": [
    {
      "filter_type_id": "uuid",
      "filter_type_name": "Work Type",
      "filter_type_slug": "work-type",
      "filter_input_type": "multi_select",
      "applies_to_category": { "category_id": "uuid", "category_name": "Saree" },
      "show_in_listing_filters": true,
      "show_in_product_details": true,
      "display_order": 1,
      "is_active": true,
      "values": [
        { "filter_value_id": "uuid", "filter_value_name": "Zari", "filter_value_slug": "zari", "is_acti
      ]
    }
  ]
}
```

`POST /api/v1/admin/filter-types`

Auth: Admin-auth

Request body:

```
{
  "filter_type_name": "Work Type",
  "filter_type_slug": "work-type",
  "filter_input_type": "multi_select",
  "applies_to_category_id": "uuid or null",
  "show_in_listing_filters": true,
  "show_in_product_details": true,
  "display_order": 1,
  "is_active": true
}
```

Validation rules:

- `filter_type_name`: required, 2–100 chars.
- `filter_type_slug`: required, unique, lowercase hyphenated.
- `filter_input_type`: required, one of `dropdown`, `multi_select`, `checkbox_group`.
- `applies_to_category_id`: optional UUID; if null, applies to both categories.
- `show_in_listing_filters`: boolean, default true.
- `show_in_product_details`: boolean, default true.

Errors:

- `409` — slug already in use.
- `422` — validation failed.

`PATCH /api/v1/admin/filter-types/:filter_type_id`

Same fields as POST, all optional.

`DELETE /api/v1/admin/filter-types/:filter_type_id`

Errors:

- `400` — filter type has values assigned to products; cascade or re-assign first.

`POST /api/v1/admin/filter-types/:filter_type_id/values`

Auth: Admin-auth

Request body:

```
{
  "filter_value_name": "Zari",
  "filter_value_slug": "zari",
  "display_order": 1,
  "is_active": true
}
```

Validation rules:

- `filter_value_name`: required, 1–100 chars.
- `filter_value_slug`: required; must be unique within this `filter_type_id`.

Constraint enforced: `UNIQUE (filter_type_id, filter_value_slug)` — prevents two `zari` values under the same filter type.

Errors:

- `409` — slug already exists under this filter type.
- `422` — validation failed.

`PATCH /api/v1/admin/filter-values/:filter_value_id`

`DELETE /api/v1/admin/filter-values/:filter_value_id`

Errors:

- `400` — value is assigned to products; remove assignments first.

Section 15 — Admin-auth: Products

`GET /api/v1/admin/products`

Auth: Admin-auth

Query params: `search`, `category_id`, `is_active`, `fabric_id`, `collection_id`, `page`, `limit`

Response `data`**:**

```
{
  "products": [
    {
      "product_id": "uuid",
      "product_name": "...",
      "product_sku": "SKU001",
      "category_name": "Saree",
      "selling_price_inr": 4500.00,
      "available_stock_quantity": 10,
      "is_active": true,
      "cover_image_url": "cdn-url",
      "created_at": "2026-01-01T00:00:00Z"
    }
  ]
}
```

`POST /api/v1/admin/products`

Auth: Admin-auth
Content-Type: `application/json`

Request body:

```
{
  "product_category_id": "uuid",
  "fabric_id": "uuid",
  "collection_id": "uuid",
  "color_id": "uuid",
  "product_name": "Kanjivaram Silk Saree",
  "product_slug": "kanjivaram-silk-saree",
  "product_sku": "SKU001",
  "product_description": "...",
  "product_care_instructions": "Dry clean only",
  "selling_price_inr": 12000.00,
```

```
  "original_price_inr": 15000.00,
  "available_stock_quantity": 5,
  "weight_in_grams": 600,
  "is_active": true,
  "is_premium_product": false,
  "is_saree_normal": true,
  "has_ready_blouse": false,
  "is_bridal_chaniya_choli": false,
  "is_sider_chaniya_choli": false,
  "seo_meta_title": "...",
  "seo_meta_description": "...",
  "filter_value_ids": ["uuid1", "uuid2"]
}
```

Validation rules:

- `product_category_id`: required, valid UUID.
- `product_name`: required, 2–200 chars.
- `product_slug`: required, unique, lowercase hyphenated.
- `product_sku`: required, unique.
- `selling_price_inr`: required, decimal >= 0.
- `original_price_inr`: optional, decimal >= `selling_price_inr` if provided.
- `available_stock_quantity`: required, integer >= 0.
- `weight_in_grams`: required, integer >= 1.
- `filter_value_ids`: optional array; all UUIDs must exist; filter values must be valid for the product's category via `applies_to_category_id`.
- Saree tags (`is_saree_normal`, `has_ready_blouse`) must only be set when `product_category_id` is Saree.
- CC tags (`is_bridal_chaniya_choli`, `is_sider_chaniya_choli`) must only be set when `product_category_id` is Chaniya Choli.

Business rule: Saving `filter_value_ids` writes rows to `product_filter_values` only; no new product columns created.

Errors:

- `409` — slug or SKU already in use.
- `422` — validation failed.

`GET /api/v1/admin/products/:product_id`

Auth: Admin-auth
Purpose: Full product detail for the admin edit form.

Response `data`**:** Same as public product detail plus admin fields and all assigned `filter_value_ids`.

`PATCH /api/v1/admin/products/:product_id`

Same fields as POST, all optional.

Errors:

- `404` — product not found.
- `409` — slug or SKU conflict.
- `422` — validation failed.

`DELETE /api/v1/admin/products/:product_id`

Errors:

- `400` — product has active or fulfilled orders; soft-delete recommended.

Section 16 — Admin-auth: Product Media Upload

`POST /api/v1/admin/products/:product_id/media`

Auth: Admin-auth
Content-Type: `multipart/form-data`
Purpose: Upload product media from admin device storage only.

Form fields:

Field	Type	Required	Notes
`files[]`	file	Yes	One or multiple files from device
`media_type`	string	Yes	`image`, `video`, or `gif` per file
`display_order`	integer	No	
`is_cover_media`	boolean	No	Default false

Validation rules:

- Only file uploads accepted. No URL or text-based media input allowed.
- Allowed MIME types for images: `image/jpeg`, `image/png`, `image/webp`.
- Allowed MIME types for videos: `video/mp4`, `video/webm`.
- Allowed MIME types for GIFs: `image/gif`.
- Max file size: 20 MB per file.

App-layer business rules (enforced in service before insert):

1. Total `image` rows for this product must not exceed 9 after upload.
2. Total `video` + `gif` rows for this product must not exceed 3 after upload.
3. If `is_cover_media = true`, all existing cover rows for the product are unset first, then new row is set as cover.
4. If no cover exists yet, the first image uploaded is automatically set as cover.

Errors:

- `400` — media limit would be exceeded (images or videos/GIFs).
- `400` — unsupported file type.
- `400` — no URL input allowed; only file uploads from device.
- `422` — missing required fields.

`PATCH /api/v1/admin/products/:product_id/media/:product_media_id`

Auth: Admin-auth
Purpose: Update display order or cover flag.

Request body:

```
{
  "display_order": 2,
  "is_cover_media": true
}
```

Business rule: Setting `is_cover_media = true` unsets the previous cover.

`DELETE /api/v1/admin/products/:product_id/media/:product_media_id`

Auth: Admin-auth

Business rule: If deleted item was the cover, backend should auto-promote the next image to cover if one exists.

Errors:

- `404` — media not found for this product.

Section 17 — Admin-auth: Homepage Content

`PATCH /api/v1/admin/announcement-strip`

Auth: Admin-auth

Request body:

```
{
  "text": "Free shipping in Gujarat!",
  "is_active": true
}
```

Validation rules:

- `text`: required if `is_active = true`, max 200 chars.

`GET /api/v1/admin/hero-slides`

Returns all slides ordered by `display_order`.

`POST /api/v1/admin/hero-slides`

Content-Type: `multipart/form-data`

Field	Type	Required
`image_file`	file	Yes
`heading`	string	Yes, max 100 chars
`subtext`	string	No, max 200 chars
`button_label`	string	Yes, max 50 chars
`target_url`	string	Yes, internal URL
`display_order`	integer	No
`is_active`	boolean	No

`PATCH /api/v1/admin/hero-slides/:hero_slide_id`

Same fields, all optional.

`DELETE /api/v1/admin/hero-slides/:hero_slide_id`

`GET /api/v1/admin/main-banners`

`PATCH /api/v1/admin/main-banners/:main_banner_id`

Content-Type: `multipart/form-data`

Field	Type	Required
`image_file`	file	No (upload new image)
`heading`	string	No
`button_label`	string	No
`target_url`	string	No
`is_active`	boolean	No

`GET /api/v1/admin/marketing-banners`

`POST /api/v1/admin/marketing-banners`

Content-Type: `multipart/form-data`

Field	Type	Required
`image_file`	file	Yes
`title`	string	Yes, max 100 chars
`description`	string	No, max 300 chars
`button_label`	string	Yes, max 50 chars
`target_url`	string	Yes
`display_order`	integer	No
`is_active`	boolean	No

`PATCH /api/v1/admin/marketing-banners/:marketing_banner_id`

`DELETE /api/v1/admin/marketing-banners/:marketing_banner_id`

`PATCH /api/v1/admin/homepage/fabrics-order`

Auth: Admin-auth
Purpose: Set which fabrics show on homepage and in which order.

Request body:

```
{
  "fabric_ids_ordered": ["uuid1", "uuid2", "uuid3"]
}
```

Validation: All UUIDs must be valid active fabrics.

`PATCH /api/v1/admin/homepage/collections-order`

Same shape as fabrics order.

Section 18 — Admin-auth: Orders

`GET /api/v1/admin/orders`

Auth: Admin-auth

Query params: `order_status`, `payment_status`, `start_date`, `end_date`, `search` (by order number or customer name), `page`, `limit`

Response `data`**:**

```
{
  "orders": [
    {
      "order_id": "uuid",
      "order_number": "SS-20260001",
      "customer_name": "Priya Shah",
      "order_status": "delivered",
      "payment_status": "paid",
      "shipping_zone": "gujarat",
      "order_grand_total_inr": 9000.00,
      "created_at": "2026-05-01T10:00:00Z",
      "delivered_at": "2026-05-08T15:00:00Z"
    }
  ]
}
```

`GET /api/v1/admin/orders/:order_id`

Auth: Admin-auth
Purpose: Full order detail for admin view.

Response `data`**:** Same as customer order detail plus full payment info, shipping snapshot, and admin notes.

`PATCH /api/v1/admin/orders/:order_id/status`

Auth: Admin-auth
Purpose: Update order status.

Request body:

```
{
  "order_status": "shipped",
  "internal_admin_notes": "Dispatched via BlueDart"
}
```

Validation rules:

- `order_status`: required, must be a valid status transition.
- When set to `delivered`, the server must record `delivered_at = now()`.

Valid status transitions:

From	Allowed next
`placed`	`processing`, `cancelled`

From	Allowed next
`processing`	`shipped`, `cancelled`
`shipped`	`delivered`
`delivered`	No change via this endpoint — cancellation handled separately

Errors:

- `400` — invalid status transition.
- `404` — order not found.

Section 19 — Admin-auth: Cancellations

`GET /api/v1/admin/cancellation-requests`

Auth: Admin-auth

Query params: `cancellation_status`, `start_date`, `end_date`, `page`, `limit`

Response `data`**:**

```
{
  "requests": [
    {
      "cancellation_request_id": "uuid",
      "order_id": "uuid",
      "order_number": "SS-20260001",
      "customer_name": "Priya Shah",
      "cancellation_status": "pending",
      "requested_at": "2026-05-09T10:00:00Z",
      "hours_since_delivery": 19
    }
  ]
}
```

`POST /api/v1/admin/cancellation-requests`

Auth: Admin-auth
Purpose: Admin logs a WhatsApp cancellation request in the system.

Request body:

```
{
  "order_id": "uuid",
  "order_item_id": "uuid or null",
  "customer_user_id": "uuid",
  "customer_request_note": "Customer messaged via WhatsApp requesting cancellation",
  "requested_at": "2026-05-09T10:00:00Z"
}
```

Validation rules:

- `order_id`: required, valid UUID.
- `customer_user_id`: required, valid UUID.
- `requested_at`: required, ISO 8601 datetime.

Business rule: `requested_at` must be within 48 hours of the order's `delivered_at`. If outside window, return `400` with a clear message.

Errors:

- `400` — request is outside the 48-hour cancellation window.
- `400` — order not in delivered status.
- `404` — order or customer not found.

`PATCH /api/v1/admin/cancellation-requests/:cancellation_request_id`

Auth: Admin-auth
Purpose: Approve or reject a cancellation request.

Request body:

```
{
  "cancellation_status": "approved",
  "admin_decision_note": "Cancellation approved; refund initiated."
}
```

Validation rules:

- `cancellation_status`: required, must be `approved` or `rejected`.
- `admin_decision_note`: optional, max 500 chars.

Business rules on approval:

1. Set `order_status = cancelled` on the related order or order item.
2. Insert a row in `review_blocks` for `(customer_user_id, product_id)` for each relevant order item, if no row already exists.
3. Do not delete or hide any existing review.

Errors:

- `400` — request already resolved.
- `404` — cancellation request not found.

Section 20 — Admin-auth: Reviews

`GET /api/v1/admin/reviews`

Auth: Admin-auth

Query params: `product_id`, `rating_value`, `is_visible`, `start_date`, `end_date`, `page`, `limit`

Response `data`**:**

```
{
  "reviews": [
    {
      "review_id": "uuid",
      "product_id": "uuid",
      "product_name": "Kanjivaram Silk Saree",
      "customer_name": "Priya Shah",
      "rating_value": 5,
      "review_title": "Beautiful saree",
      "review_text": "...",
      "is_visible": true,
      "created_at": "2026-05-10T10:00:00Z"
    }
  ]
}
```

`PATCH /api/v1/admin/reviews/:review_id/visibility`

Auth: Admin-auth
Purpose: Hide or unhide a review for abuse or legal reasons only.

Request body:

```
{ "is_visible": false }
```

Validation rules:

- `is_visible`: required, boolean.

Business rule: Admin may only hide clearly abusive or illegal content. Reviews must not be auto-deleted or removed due to cancellations.

Errors:

- `404` — review not found.

Section 21 — Admin-auth: Shipping Rates

`GET /api/v1/admin/shipping-rates`

Auth: Admin-auth

Response `data`**:**

```
{
  "shipping_rates": [
    {
      "shipping_rate_id": "uuid",
      "shipping_zone": "india",
      "rate_per_gram_inr": 0.0050,
      "minimum_shipping_charge_inr": 99.00,
      "is_active": true
    }
  ]
}
```

`POST /api/v1/admin/shipping-rates`

Auth: Admin-auth

Request body:

```
{
  "shipping_zone": "india",
  "rate_per_gram_inr": 0.0050,
  "minimum_shipping_charge_inr": 99.00,
  "is_active": true
}
```

Validation rules:

- `shipping_zone`: required, must be `india` or `international`. Gujarat is free by business rule and has no rate row.
- `rate_per_gram_inr`: required, decimal >= 0.
- `minimum_shipping_charge_inr`: required, decimal >= 0.

Errors:

- `409` — active rate for this zone already exists.
- `422` — validation failed.

`PATCH /api/v1/admin/shipping-rates/:shipping_rate_id`

Same fields, all optional.

Section 22 — Admin-auth: Site Settings

`GET /api/v1/admin/site-settings`

Auth: Admin-auth

Response `data`**:**

```
{
  "settings": [
    { "setting_key": "whatsapp_number", "setting_type": "string", "setting_value": "+919723421411" },
    { "setting_key": "premium_price_threshold", "setting_type": "decimal", "setting_value": "5000.00" },
    { "setting_key": "currency", "setting_type": "string", "setting_value": "INR" }
  ]
}
```

`PATCH /api/v1/admin/site-settings/:setting_key`

Auth: Admin-auth

Request body:

```
{ "setting_value": "7000.00" }
```

Validation rules:

- `setting_value`: required, string.
- Backend validates value against `setting_type`:
  - `string`: accept as-is after trim, max 500 chars.
  - `decimal`: must parse to a valid non-negative decimal.
  - `integer`: must parse to a valid non-negative integer.
  - `boolean`: must be `"true"` or `"false"` string.
- If `premium_price_threshold` is not a valid decimal, reject with `422`.

Errors:

- `400` — value fails type validation.
- `404` — setting key not found.
- `422` — validation failed.

Optional refinements for later consideration

- Add `GET /api/v1/products/fabrics/:fabric_slug` and `GET /api/v1/products/collections/:collection_slug` as shortcut listing URLs for fabric/collection cards.
- Add `POST /api/v1/auth/forgot-password` and `POST /api/v1/auth/reset-password` for password recovery flow.
- Add `GET /api/v1/states` as a public endpoint for populating address state dropdowns on the frontend.

- Add rate limiting to auth endpoints to prevent brute-force.
- Razorpay webhook handler endpoint `POST /api/v1/payments/razorpay/webhook` for server-to-server payment events.

Synchronization / Consistency Matrix

This section adds a practical consistency plan to the API and backend design. The goal is balanced protection: use strong guarantees only where shared mutable state can break, and keep read-heavy or low-risk flows simple.

Protection types used

Protection type	Meaning
No special synchronization needed	Normal validation and standard DB reads/writes are enough
Unique constraint + retry-safe behavior	Prevent duplicates with DB uniqueness and treat repeated requests safely
Idempotency	Same logical request repeated returns the same result instead of creating duplicates
Optimistic concurrency	Prevent silent overwrite by requiring client to send the version/ETag it last read
Transaction / atomic update	Multiple related writes succeed or fail together, with correct final state under concurrency

Extra schema / protocol support to add

Item	Purpose
`idempotency_keys` table	Store idempotent POST request fingerprints and final responses for retry-safe create/finalize flows
`updated_at` on all admin-editable tables	Base field for optimistic concurrency
Optional `version_number` integer on admin-editable tables	Alternative to `updated_at`; increments each successful update
`ETag` or `resource_version` response field	Returned by admin read endpoints so frontend can send `If-Match` or version on update
Unique index on `wishlist_items (customer_user_id, product_id)`	Prevent duplicate wishlist rows
Unique index on `cart_items (cart_id, product_id)`	Prevent duplicate cart rows for same product
Unique index on `reviews (customer_user_id, product_id)`	Prevent duplicate reviews
Unique index on `review_blocks (customer_user_id, product_id)`	Prevent duplicate review-block rows
Partial unique index for product cover media	Enforce at most one cover media per product
Unique index on `idempotency_keys (scope, actor_id, idempotency_key)`	Ensure one logical create/finalize operation per actor and scope
Unique constraint on pending-order reuse key	Prevent multiple equivalent pending orders during retry window

Suggested `idempotency_keys` table

Column	Type	Notes
`idempotency_key_id`	UUID	Primary key
`scope`	string	Example: `checkout_create_order`, `payment_verify`, `admin_cancellation_decision`
`actor_type`	string	`customer` or `admin`
`actor_id`	UUID	Customer or admin user id
`idempotency_key`	string	Client-supplied key
`request_hash`	string	Hash of canonical request payload for same-key mismatch detection

Column	Type	Notes
`request_method`	string	Usually POST
`request_path`	string	Endpoint path
`status`	string	`in_progress`, `completed`, `failed`
`response_status_code`	integer	Stored original response code
`response_body_json`	jsonb	Stored original full JSON response
`created_at`	timestamp	
`expires_at`	timestamp	TTL for safe reuse

Suggested optimistic concurrency contract

For admin update endpoints that need overwrite protection:

- Read endpoints return one of:
  - `etag` field in `data`, or
  - `resource_version`, or
  - `updated_at`
- Update endpoints require either:
  - `If-Match` header with ETag, or
  - `resource_version` in JSON body
- If server version does not match client version, return:
  - `412 Precondition Failed` when using `If-Match`, or
  - `409 Conflict` if using explicit version field

Recommended choice: use `If-Match` + ETag for admin PATCH endpoints because it cleanly separates concurrency control from business fields.

Matrix by flow

Flow / endpoint group	Risk	Protection type	Why it is needed	Schema / index support
Public homepage fetch, listings, search, product detail	Read-only; stale cache is acceptable	No special synchronization needed	No shared mutable state is changed	Normal caching only
Customer order history reads	Read-only	No special synchronization needed	Viewing data cannot corrupt state	None
Customer wishlist reads	Read-only	No special synchronization needed	Pure read flow	None
Admin dashboard/report reads	Read-only aggregates	No special synchronization needed	Reporting can tolerate normal read consistency	Optional cache/materialized reporting later
Fabrics/colors/collections simple reads	Read-only	No special synchronization needed	No conflicting writes in read flow	None
Wishlist add `POST /wishlist`	Double click / retry could create duplicate row	Unique constraint + retry-safe behavior	Duplicate adds should not create duplicate wishlist items; same product should exist only once	Unique index `(customer_user_id, product_id)`

Flow / endpoint group	Risk	Protection type	Why it is needed	Schema / index support
Wishlist remove `DELETE /wishlist/:product_id`	Retry after first delete	Unique constraint + retry-safe behavior	Delete should be safe to repeat and can return success even if already absent, or 404 by contract if preferred	Same unique index
Cart add `POST /cart/items`	Double click could create duplicate cart rows	Unique constraint + normal validation + retry-safe merge behavior	Same product should merge into one cart line, not duplicate rows	Unique index `(cart_id, product_id)`
Cart update `PATCH /cart/items/:cart_item_id`	Retries / race between same user actions	Normal validation + retry-safe set behavior	Quantity update is deterministic if treated as set-to-value, not increment blindly	Same cart unique index
Cart remove / clear	Retry after first success	Retry-safe normal behavior	Repeating delete/clear should be harmless	None beyond cart ownership check
Guest cart merge into customer cart	Same product in both carts	Deterministic merge rule, no lock usually needed	Must avoid duplicate rows and unpredictable quantity results	Unique index `(cart_id, product_id)`
Checkout create-order `POST /checkout/create-order`	Double click / mobile retry could create multiple pending orders	Idempotency + transaction	Same logical checkout attempt should return same pending order within reuse window; order snapshot creation touches several tables	`idempotency_keys`; optional `pending_order_signatur` unique support
Payment verify `POST /payments/razorpay/verify`	Duplicate callback, retry, double finalization	Strict idempotency + transaction	Payment success processing must run once; repeated verify must return same result, not double-finalize order	`idempotency_keys`; unique provider payment id; transaction o order/payment/stock
Final stock deduction at payment finalization	Two buyers compete for low stock	Transaction / atomic update	Cart does not reserve stock; only finalization must safely decrement available stock once	Atomic stock update query or row lock on product rows in transaction
Review create `POST /products/:product_id/reviews`	Double submit / duplicate review	Unique constraint + validation + retry-safe create	Only one review per customer per product; eligibility and review block checks are needed, but no heavy locking	Unique index `(customer_user_id, product_id)`; unique `review_blocks` index
Customer profile edit	Usually single-user self-edit	No special synchronization needed	Last-write-wins is acceptable here	None
Customer address add/edit/delete	Usually single-user self-edit	No special synchronization needed	Low risk; shipping should always recalc from latest saved data	None

Flow / endpoint group	Risk	Protection type	Why it is needed	Schema / index support
Admin product edit PATCH `/admin/products/:product_id`	Two admins can overwrite each other silently	Optimistic concurrency	Product has many fields and assignments; stale edit must be rejected	`updated_at` or `version_number`; ETag/If-Match
Admin filter type edit	Two admins edit same filter config	Optimistic concurrency	Prevent silent overwrite of filter behavior and visibility	`updated_at` or `version_number` on `filter_types`
Admin filter value create	Duplicate value slug under same filter type	Unique constraint + normal validation	Prevent duplicate value identity without locking	Unique (`filter_type_id`, `filter_value_slug`)
Admin filter value edit	Two admins edit same value	Optimistic concurrency	Protect label/slug/visibility from silent overwrite	`updated_at` on `filter_values`
Admin homepage hero/banners/announcement edits	Two admins overwrite config	Optimistic concurrency	Homepage content and display order can easily be lost by stale saves	`updated_at` or version on content tables
Admin homepage fabrics order / collections order	Two admins reorder concurrently	Optimistic concurrency	Array/order updates are especially vulnerable to silent overwrite	Version on homepage-config row or ordered mapping table
Admin shipping rates edit	Conflicting admin edits on rate/minimum charge	Optimistic concurrency	Financial config must not silently lose edits	`updated_at` / ETag on `shipping_rates`
Admin site settings edit	Conflicting admin edits on WhatsApp number or threshold	Optimistic concurrency	Important mutable settings should reject stale saves	`updated_at` / ETag on `site_settings`
Admin product media upload	Same time uploads	No strict lock for upload itself	File upload can remain simple if limits are validated per request	Existing media counts checked at write time
Admin media cover switch PATCH `/products/:product_id/media/:product_media_id`	Two cover toggles can leave invalid final state	Transaction / atomic update	Need at most one cover media per product in final state	Partial unique cover index + transaction to unset old/set new
Admin cancellation request create	Duplicate logging of same WhatsApp cancellation	Unique constraint + normal validation, optional idempotency	Prevent duplicate pending requests for same order or order item while pending	Unique pending-request constraint
Admin cancellation approve/reject	Double click / retry / simultaneous decision	Idempotent admin action + transaction + unique constraint	Same decision should not apply twice; approval updates order and inserts review block safely	`idempotency_keys`; unique `review_blocks`; transaction
Admin manual order status update	Invalid regressions or stale admin page	Status transition validation + optimistic concurrency	Prevent both illegal transitions and stale overwrite	`updated_at`/ETag on orders

Flow / endpoint group	Risk	Protection type	Why it is needed	Schema / index support
Admin manual payment-status update	Conflicting manual changes	Optimistic concurrency + status validation	Avoid stale overwrite and impossible status combinations	`updated_at`/ETag on orders/payments

Practical rules by flow

1) Read-only browsing flows

Endpoints:

- `GET /api/v1/home`
- `GET /api/v1/products`
- `GET /api/v1/products/sarees`
- `GET /api/v1/products/chaniya-cholis`
- `GET /api/v1/products/:product_slug`
- `GET /api/v1/search`
- `GET /api/v1/account/orders`
- admin report/dashboard reads

Classification: No special synchronization needed.

Notes:

- Standard response caching is fine.
- Product stock shown on UI is informational only and may become stale before checkout.
- No locks, no version checks needed for reads.

2) Wishlist

Endpoints:

- `POST /api/v1/wishlist`
- `DELETE /api/v1/wishlist/:product_id`

Classification: Unique constraint + idempotent-style behavior.

Backend rules:

- Add uses uniqueness on `(customer_user_id, product_id)`.
- Repeated add should return success and the existing row, not fail noisily, if desired for frontend smoothness.
- Delete should be retry-safe; repeated delete may either:
  - return `200` with `removed: false`, or
  - keep current `404` contract.

Recommended schema:

```
UNIQUE (customer_user_id, product_id)
```

No row-level locking needed.

3) Cart

Endpoints:

- `POST /api/v1/cart/items`
- `PATCH /api/v1/cart/items/:cart_item_id`
- `DELETE /api/v1/cart/items/:cart_item_id`
- `DELETE /api/v1/cart`
- guest cart merge on login

Classification: Mostly normal validation + unique cart-item rule + retry-safe behavior.

Backend rules:

- One cart line per `(cart_id, product_id)`.
- `POST /cart/items` should merge if row already exists:
  - recommended behavior: increase quantity by requested amount, capped by stock at that moment, or reject if over stock.
- `PATCH /cart/items/:cart_item_id` should be set-to-quantity, not increment-by-delta.
- `DELETE` and clear-cart should be safe to repeat.

Guest cart merge rule:

- On login, merge guest cart into customer cart by product.
- Deterministic rule:
  - if product exists in both carts, new quantity = sum of both quantities
  - then clamp to latest available stock or reject with clear partial-merge response
- Prefer deterministic server merge; frontend should never guess.

Recommended schema:

```
UNIQUE (cart_id, product_id)
```

No strict locking needed for normal cart operations because cart does not reserve stock.

4) Checkout create-order

Endpoint:

- `POST /api/v1/checkout/create-order`

Classification: Idempotency + transaction.

Why:

- Double click or retry could create multiple pending orders for one checkout intent.
- The flow snapshots cart items, shipping, address, measurements, and creates a provider order reference.

Contract addition:

- Require header: `Idempotency-Key: <uuid>`
- Scope is per customer and endpoint.

Backend behavior:

1. Canonicalize request payload and hash it.
2. Check `idempotency_keys` for `(scope='checkout_create_order', actor_id=customer_id, idempotency_key)`.
3. If completed, return stored original response.

4. If in progress, return `409 Conflict` with retry message.
5. If same key is reused with different request payload, return `409 Conflict` or `422` key mismatch.
6. If not found, create idempotency row and execute order creation inside transaction.
7. Store final response JSON and status code in `idempotency_keys`.

Pending-order reuse window:

- Same key should return same pending order for a safe window, for example 24 hours.
- Optional enhancement: also prevent accidental duplicate pending orders using a `pending_order_signature` built from customer + cart fingerprint + address + measurements + amount.

Schema additions:

- `idempotency_keys`
- Optional `orders.checkout_request_hash`
- Optional partial unique index for unresolved pending order signature

5) Payment verification

Endpoint:

- `POST /api/v1/payments/razorpay/verify`

Classification: Strict idempotency + transaction.

Why:

- The payment gateway callback, frontend retry, and duplicate click can all hit the endpoint more than once.
- Finalization changes order state, payment state, stock, and cart.

Contract addition:

- Require `Idempotency-Key` header from frontend verify call.
- Also enforce uniqueness on provider payment identifiers, especially `razorpay_payment_id`.

Backend behavior:

1. Lookup or create idempotency record for scope `payment_verify`.
2. Verify request payload hash matches prior request for same key.
3. Inside one DB transaction:
   - load target order
   - verify order still in payable/pending state
   - verify Razorpay signature
   - ensure `razorpay_payment_id` has not already been processed for another finalized payment
   - perform stock deduction atomically
   - set payment row to paid
   - set order status to placed
   - clear cart
4. Commit and store exact response under idempotency key.
5. Retry returns exact same response body/status.

Recommended constraints:

```
UNIQUE (payment_gateway_name, gateway_payment_id)
UNIQUE (scope, actor_id, idempotency_key)
```

Important note: If first attempt is still in progress, return `409 Conflict` and ask client to retry later.

6) Final stock deduction

Flow: payment finalization only

Classification: Transaction / atomic update.

Why:

- Cart does not reserve inventory.
- Stock displayed on listing/PDP/cart can become stale.
- Only final order finalization must decide the winner under contention.

Recommended backend pattern:

- Inside payment verification transaction, for each order item either:
  - lock product row (`SELECT ... FOR UPDATE`) then validate + decrement, or
  - run atomic conditional update such as "decrease stock where stock >= needed quantity" and check affected row count.
- If any item fails stock check, rollback the whole finalization and return out-of-stock / insufficient-stock result.

Why not optimistic locking here:

- This is not an editor-overwrite problem; it is a shared counter / inventory race.
- Atomic stock decrement is the correct protection.

7) Reviews

Endpoint:

- `POST /api/v1/products/:product_id/reviews`

Classification: Unique constraint + validation + retry-safe create.

Why:

- Double submit should not create duplicate reviews.
- Eligibility comes from business checks, not heavy locking.

Backend rules:

- Validate delivered order item ownership.
- Validate no `review_blocks` entry exists.
- Validate no existing review for same `(customer_user_id, product_id)`.
- Attempt insert; if duplicate unique conflict occurs because of race, return `409 Review already exists` or return existing review.

Recommended constraints:

```
UNIQUE (customer_user_id, product_id) on reviews
UNIQUE (customer_user_id, product_id) on review_blocks
```

No strict row locks needed.

8) Customer profile and address edits

Endpoints:

- `PATCH /api/v1/account/profile`
- `POST/PATCH/DELETE /api/v1/account/addresses`

Classification: No special synchronization needed.

Why:

- These are normally self-service edits by one user.
- Last-write-wins is acceptable.
- Checkout always recalculates shipping and totals from latest persisted address and rate data.

Optional note: If address-book conflicts become common later, optimistic concurrency can be added without changing business rules.

9) Admin product edit

Endpoint:

- `PATCH /api/v1/admin/products/:product_id`

Classification: Optimistic concurrency.

Why:

- Products have many editable fields, tags, SEO fields, filters, and stock metadata.
- Two admins can silently overwrite each other if stale edit forms are saved.

Contract update:

- `GET /api/v1/admin/products/:product_id` returns `etag` or `resource_version`.
- `PATCH` requires `If-Match: <etag>` or `resource_version` in body.
- If mismatch, return `412 Precondition Failed` with latest server version metadata.

Schema support:

- `products.updated_at` already useful; optionally add `version_number`.

10) Admin filter types / values

Endpoints:

- `PATCH /api/v1/admin/filter-types/:filter_type_id`
- `POST /api/v1/admin/filter-types/:filter_type_id/values`
- `PATCH /api/v1/admin/filter-values/:filter_value_id`

Classification:

- Filter type edit: Optimistic concurrency
- Filter value create: Unique constraint + normal validation
- Filter value edit: Optimistic concurrency

Why:

- Filter-type visibility and behavior can affect listing filters and PDP attributes globally.
- Value identity must stay unique inside its parent filter type.

Constraints:

```
UNIQUE (filter_type_id, filter_value_slug)
```

11) Admin homepage content and ordering

Endpoints:

- `PATCH /api/v1/admin/announcement-strip`
- `PATCH /api/v1/admin/main-banners/:main_banner_id`
- `PATCH /api/v1/admin/homepage/fabrics-order`
- `PATCH /api/v1/admin/homepage/collections-order`
- hero slide PATCH / reorder operations

Classification: Optimistic concurrency.

Why:

- Reorder arrays and homepage config are classic silent-overwrite risk areas.
- One admin may reorder while another edits labels or activation state.

Contract update:

- Read endpoints return `etag` for the collection/config resource.
- Save/reorder endpoints require `If-Match`.
- Conflict returns `412 Precondition Failed`.

Schema support options:

- `homepage_sections.updated_at` per table, or
- one `homepage_config` aggregate row with a version for order arrays.

12) Admin shipping rates

Endpoints:

- `PATCH /api/v1/admin/shipping-rates/:shipping_rate_id`

Classification: Optimistic concurrency.

Why:

- Financial config changes should not be silently overwritten.

Contract update:

- Return `etag` on GET.
- Require `If-Match` on PATCH.

13) Admin site settings

Endpoints:

- `PATCH /api/v1/admin/site-settings/:setting_key`

Classification: Optimistic concurrency.

Why:

- WhatsApp number, premium threshold, and currency are important live settings.
- Silent overwrite is unacceptable.

Contract update:

- `GET /api/v1/admin/site-settings` returns per-setting `etag` or `updated_at`.
- `PATCH` requires `If-Match`.

14) Product media management

Endpoints:

- `POST /api/v1/admin/products/:product_id/media`
- `PATCH /api/v1/admin/products/:product_id/media/:product_media_id`
- `DELETE /api/v1/admin/products/:product_id/media/:product_media_id`

Classification:

- Upload: No strict locking needed
- Cover switch: Transaction / atomic update

Why:

- File upload itself is low-risk.
- Cover state is shared mutable state and must end with at most one cover media row per product.

Recommended DB support:

- Partial unique index in PostgreSQL:

```
UNIQUE (product_id) WHERE is_cover_media = true
```

Backend rule for cover switch:

- In one transaction:
  1. unset current cover for product
  2. set target row as cover
- If target media is not for the product, reject.

Delete rule:

- If cover is deleted, promote next eligible image atomically.

15) Cancellation approval / rejection

Endpoints:

- `POST /api/v1/admin/cancellation-requests`
- `PATCH /api/v1/admin/cancellation-requests/:cancellation_request_id`

Classification:

- Create request: Unique constraint + validation, optional idempotency
- Approve/reject: Idempotent admin action + transaction + unique constraint

Why:

- Same WhatsApp request should not be logged multiple times as separate pending requests.
- Approving twice must not double-cancel or duplicate review blocks.

Recommended constraints:

- If only one open cancellation per order item is allowed:

```
UNIQUE (order_item_id, cancellation_status='pending')   [implemented as partial unique index where supp
```

- Or for whole-order cancellation requests:

```
UNIQUE (order_id, cancellation_status='pending')
```

- For review block safety:

```
UNIQUE (customer_user_id, product_id) on review_blocks
```

Approve/reject backend behavior:

- Require `Idempotency-Key` on approval/rejection action, or make the state transition itself idempotent by checking current resolved status.
- Inside one transaction:
  1. load cancellation request
  2. ensure still pending
  3. set approved/rejected
  4. if approved, update order/order item status as designed
  5. insert review_blocks with upsert/do-nothing semantics
- Retry returns same resolved outcome.

16) Admin order status / payment status manual updates

Endpoints:

- `PATCH /api/v1/admin/orders/:order_id/status`
- any manual payment-status PATCH if retained

Classification: Status transition validation + optimistic concurrency.

Why:

- Orders can be edited by multiple admins.
- Beyond stale overwrite, invalid status rollback/regression must be blocked.

Rules:

- Keep the status transition matrix already defined.
- Require `If-Match` on manual admin updates.
- Return `412 Precondition Failed` if another admin updated the order after the page was loaded.

No heavy lock needed for standard manual status change, because this is an editor consistency problem, not a stock-counter race.

API contract updates to add

New common headers

For selected POST endpoints with idempotency:

```
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

Required for:

- `POST /api/v1/checkout/create-order`
- `POST /api/v1/payments/razorpay/verify`
- Recommended for `PATCH /api/v1/admin/cancellation-requests/:cancellation_request_id` if implemented as command-style action

Rules:

- Key must be unique per logical operation.
- Same key + same payload returns original response.
- Same key + different payload returns `409 Conflict` or `422 Unprocessable Entity`.
- If original request still processing, return `409 Conflict` with retry message.

New common headers for optimistic concurrency

For selected admin PATCH endpoints:

```
If-Match: "etag-value-here"
```

Required for:

- `PATCH /api/v1/admin/products/:product_id`
- `PATCH /api/v1/admin/filter-types/:filter_type_id`
- `PATCH /api/v1/admin/filter-values/:filter_value_id`
- `PATCH /api/v1/admin/announcement-strip`
- `PATCH /api/v1/admin/main-banners/:main_banner_id`
- hero/banner/homepage reorder endpoints
- `PATCH /api/v1/admin/shipping-rates/:shipping_rate_id`
- `PATCH /api/v1/admin/site-settings/:setting_key`
- `PATCH /api/v1/admin/orders/:order_id/status`
- manual payment-status update endpoint, if used

Conflict response example:

```
{
  "success": false,
  "message": "Resource was updated by another admin. Refresh and try again.",
  "errors": {
    "concurrency": ["ETag mismatch"]
  },
  "meta": {
    "current_etag": "new-server-etag"
  }
}
```

Recommended HTTP code:

- `412 Precondition Failed` for `If-Match` mismatch

Extra constraints and indexes to add

Recommended uniqueness and partial uniqueness

```
wishlist_items: UNIQUE (customer_user_id, product_id)
cart_items: UNIQUE (cart_id, product_id)
reviews: UNIQUE (customer_user_id, product_id)
review_blocks: UNIQUE (customer_user_id, product_id)
filter_values: UNIQUE (filter_type_id, filter_value_slug)
idempotency_keys: UNIQUE (scope, actor_id, idempotency_key)
payments: UNIQUE (payment_gateway_name, gateway_payment_id)
```

Recommended partial unique indexes where supported

```
product_media: UNIQUE (product_id) WHERE is_cover_media = true
cancellation_requests: one pending request per order or order item
orders: optional one unresolved pending order per checkout signature within reuse window
```

Final design stance

This synchronization plan is intentionally selective:

- **No heavy locking** on browsing, reads, wishlist reads, normal profile edits, or ordinary cart interactions.
- **Uniqueness + retry-safe behavior** for wishlist, cart line identity, and reviews.
- **Idempotency** for create-order, payment verification, and optionally command-like admin approval actions.
- **Optimistic concurrency** for admin-edited resources where silent overwrite is the main real-world risk.
- **Transactions / atomic updates** only for truly shared mutable state: payment finalization, stock deduction, cover media switching, and cancellation approval side effects.

This keeps the system safe where it can actually break, without over-engineering simple reads and low-risk user actions.